# 基于简历的面试 Q&A（分级）

## 初级

**Q1：你最近一份工作的核心职责是什么？**
A：在河南产权交易中心，负责多个独立系统的需求分析、架构设计与落地，包括专家培训考试、电子发票网关、金融保函等；独立完成接口设计与文档编写，配合前端联调测试与上线运维。

**Q2：你做过哪些 .NET 技术栈的项目？**
A：涵盖 .NET Framework、.NET Core、ASP.NET MVC、ASP.NET Core Web API，同时有 WinForms/WPF 桌面端和 Vue3 中后台项目经验。

**Q3：Redis 在你的项目中用过哪些场景？**
A：缓存、限流、Pub/Sub、队列等；比如登录防刷限流、WebSocket 跨节点消息路由。

```
// 计数器限流（示例）
var key = $"login:rate:{phone}";
var count = await db.StringIncrementAsync(key);
if (count == 1) await db.KeyExpireAsync(key, TimeSpan.FromMinutes(1));
if (count > 5) throw new Exception("Too many requests");
```

**Q4：你常用哪些 ORM？如何选择？**
A：EF Core、Dapper、SqlSugar。复杂模型用 EF Core，性能敏感或需精细 SQL 控制用 Dapper，快速 CRUD 和复杂查询封装可用 SqlSugar。

```
// EF Core
var user = await db.Users.FirstOrDefaultAsync(x => x.Id == id);

// Dapper
var user2 = await conn.QuerySingleAsync<User>(
    "select * from Users where Id=@id", new { id });
```

**Q5：你如何保障接口文档质量？**
A：在需求阶段定义接口规范，明确参数、返回、错误码与示例；联调中补全日志与异常说明。

## 中级

**Q6：专家培训与考试系统的高并发登录怎么做的？**
A：使用 Redis 滑动窗口限流，结合短信登录防刷策略；峰值期开考登录稳定率保持 99%+。

```
// Redis ZSET 滑动窗口（示例）
var now = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
var key = $"sms:login:{phone}";
await db.SortedSetRemoveRangeByScoreAsync(key, 0, now - 60_000);
await db.SortedSetAddAsync(key, now, now);
await db.KeyExpireAsync(key, TimeSpan.FromMinutes(1));
var count = await db.SortedSetLengthAsync(key);
if (count > 5) throw new Exception("Too many requests");
```

**Q7：文件转码链路如何设计？**
A：采用 Redis 队列 + Webhook 构建异步转码管道，任务异步化避免阻塞主业务，平均等待时长下降约 40%。

```
// 入队转码任务
await db.ListRightPushAsync("transcode:queue", JsonSerializer.Serialize(job));

// 消费（示例）
var item = await db.ListLeftPopAsync("transcode:queue");
if (!item.IsNullOrEmpty)
{
    // 调用 FFmpeg 或转码服务
}
```

**Q8：七牛云分片直传与断点续传的关键点？**
A：分片上传凭证 + 服务端合并；断点续传通过记录已上传分片和文件 hash 校验，弱网环境上传成功率提升约 30%。

```
// 记录已上传分片（示例）
var partKey = $"upload:{fileHash}:parts";
await db.SetAddAsync(partKey, partIndex);
await db.KeyExpireAsync(partKey, TimeSpan.FromHours(6));
```

**Q9：多节点 WebSocket 消息路由如何做一致性？**
A：使用 Redis Pub/Sub 进行跨节点广播，各节点订阅同一通道并按连接路由转发。

```
// 发布
await sub.PublishAsync("ws:broadcast", payload);

// 订阅
await sub.SubscribeAsync("ws:broadcast", (ch, msg) =>
{
    // 根据连接路由转发
});
```

**Q10：RBAC 与多租户隔离如何落地？**
A：RBAC 采用角色-权限-用户模型；多租户在数据库层以租户字段隔离，服务层统一鉴权拦截，确保数据隔离。

```
-- 租户隔离查询
SELECT * FROM Orders WHERE TenantId = @tenantId;
```

**Q11：你如何做线上问题定位？**
A：先看日志与监控指标定位范围，再结合请求链路排查 SQL、外部接口、网络与资源瓶颈。

## 高级

**Q12：电子发票网关为何采用工厂模式？**
A：第三方开票渠道差异大，用工厂模式做统一抽象，新增服务商只需实现统一接口并注册即可，做到“即插即用”。

```
public interface IInvoiceClient
{
    Task<string> IssueAsync(InvoiceRequest req);
}

public sealed class InvoiceClientFactory
{
    public IInvoiceClient Create(string vendor) => vendor switch
```

```
            {
                "A" => new VendorAClient(),
                "B" => new VendorBClient(),
                _ => throw new NotSupportedException()
            };
        }
```

**Q13：MongoDB 缓存第三方接口数据为什么用 TTL？**
A：第三方数据需定时更新，TTL 自动过期减少手动清理成本，并降低第三方 API 调用次数。

```
        // TTL 索引：到期自动删除
        db.thirdPartyCache.createIndex({ expireAt: 1 }, { expireAfterSeconds: 0 })
```

**Q14：HDIS 报表查询性能优化怎么做？**
A：分析慢 SQL 并针对高频条件建索引；结合 Redis 缓存，响应从约 1000ms 降到约 200ms。

```
        -- 针对报表的复合索引
        CREATE INDEX IX_Report_Department_Date
        ON DialysisReport(DepartmentId, ReportDate DESC);
```

```
        // 报表缓存（示例）
        var cacheKey = $"report:{deptId}:{date:yyyyMMdd}";
        var cached = await db.StringGetAsync(cacheKey);
        if (!cached.IsNullOrEmpty) return cached;
```

**Q15：Workflow Core 审批流的设计要点？**
A：基于部门/角色/身份配置权限，节点配置化；支持多级流转与动态流程，提交后自动进入审批链路。

```
        public class ApprovalWorkflow : IWorkflow<ApprovalData>
        {
            public void Build(IWorkflowBuilder<ApprovalData> builder)
            {
                builder
                    .StartWith<SubmitStep>()
                    .Then<ManagerApproveStep>()
                    .If(data => data.Amount > 10000).Do(then =>
then.Then<DirectorApproveStep>())
                    .Then<FinishStep>();
            }
        }
```

**Q16：你在跨网闸与多分院部署中怎么保证一致性？**
A：依赖控制与镜像版本一致，使用 Helm 编排保证部署一致性，结合 Jenkins 流水线规范化发布。